

Décisions Architecturales

L'une des attentes fondamentales envers un architecte est de prendre des décisions d'architecture.

Les décisions d'architecture concernent généralement la structure de l'application ou du système, mais elles peuvent aussi porter sur des choix technologiques, en particulier lorsque ces choix ont un impact sur les caractéristiques architecturales.

Quel que soit le contexte, une bonne décision d'architecture est une décision qui aide à orienter les équipes de développement vers les bons choix techniques.

Prendre des décisions d'architecture implique de recueillir suffisamment d'informations pertinentes, de justifier la décision, de la documenter et de la communiquer efficacement aux bonnes parties prenantes.

Les anti-patterns de prise de décision architecturale

Des modes d'échec dans la manière dont les équipes prennent, justifient, documentent et communiquent les décisions d'architecture.

En architecture, la décision elle-même est un artefact central. Si le processus décisionnel est mauvais, le code finit généralement par refléter ce mauvais processus.

Un cadrage utile est le suivant : une décision d'architecture compte lorsqu'elle est **architecturalement significative**, c'est-à-dire lorsqu'elle affecte la structure du système, ses qualités non fonctionnelles, ses dépendances, ses interfaces ou ses techniques de construction.

L'idée des ADR (*Architecture Decision Records*) existe précisément pour garder une trace durable du **pourquoi**, pas seulement du choix final. Sans cela, les gens appliquent les décisions à l'aveugle ou les remettent en cause à l'aveugle, et dans les deux cas c'est dangereux.

1. The Covering Your Assets Antipattern

Cela se produit lorsqu'un architecte ou un lead technique retarde ou évite une décision par peur de se tromper ou d'être tenu responsable plus tard. Le résultat n'est pas la prudence, mais une indécision déguisée en prudence

- Pourquoi c'est problématique ?
Parce qu'en architecture, beaucoup de décisions ne peuvent pas être reportées indéfiniment sans coût.
- Exemple
Monolithe modulaire ou microservices ? Appels synchrones autorisés ou non entre *bounded contexts* ? PostgreSQL comme source transactionnelle de vérité ? Consistance éventuelle acceptable ou non ? Isolation multi-tenant au niveau schéma ou base de données ? Si personne ne tranche, les développeurs comblent le vide localement. L'architecture devient alors accidentelle au lieu d'être intentionnelle. Le résultat typique, c'est de l'incohérence, des débats répétés, des implémentations incompatibles et des retards.

La bonne réponse n'est pas de décider tout trop tôt. Ce serait une autre erreur. Le principe pertinent est de décider au **dernier moment responsable** : assez tard pour disposer d'informations suffisantes, mais pas si tard que cela bloque la livraison ou fasse tomber l'équipe dans l'analyse sans fin.

- Exemple
Une équipe construit une plateforme de planification hospitalière et doit décider s'il faut utiliser une intégration orientée événements entre planification, facturation et notifications. Un architecte en mode **covering your assets** dit pendant six semaines : « on verra plus tard, ça dépend ». Pendant ce temps, une équipe utilise du REST direct, une autre introduit un broker de messages, et une troisième écrit des scripts de réconciliation (cron-job reconciliation).
 - Résultat : trois modèles d'intégration différents dans le même système. Le vrai problème n'était pas la technique, mais l'absence de décision.

2. Groundhog Day Antipattern

Ici, une décision a bien été prise, mais la justification est absente, incomplète ou mal formulée. Donc le même sujet revient encore et encore, parce que personne ne sait réellement **pourquoi** le choix a été fait.

Le nom est très parlant. Toutes les quelques semaines, un nouveau membre rejoint l'équipe, une contrainte évolue, ou une discussion de sprint rouvre le sujet, et soudain on recommence : « Pourquoi pas GraphQL plutôt que REST ? », « Pourquoi n'a-t-on pas découpé ce module en microservices ? », « Pourquoi avons-nous choisi Kafka ? ». Si la seule réponse est « parce que c'était décidé » ou « parce que l'architecte l'a dit », alors il n'existe pas de raisonnement exploitable. Ce n'est pas de la gouvernance, c'est de l'amnésie institutionnelle.

La cause racine est souvent que la décision n'a qu'une justification **technique** ou qu'une justification **métier**, mais pas les deux.

Une bonne décision d'architecture doit généralement expliciter les deux dimensions. « Nous avons choisi PostgreSQL parce qu'il nous faut une forte cohérence transactionnelle et des jointures complexes » est une justification technique. « Nous l'avons choisi parce que les erreurs de réconciliation financière coûtent cher et que des rapports tardifs pénaliseraient les opérations » est une justification métier.

- Exemple
L'équipe décide de garder un monolithe modulaire au lieu de passer aux microservices. La raison technique est que l'indépendance de déploiement n'est pas encore nécessaire et que les transactions distribuées ajouteraient une complexité inutile. La raison métier est que le produit cherche encore son *market fit*, donc la vitesse de livraison et la faible charge opérationnelle comptent plus que l'autonomie fine des services. Si ces deux raisons ne sont pas documentées, un nouvel ingénieur senior va rouvrir le débat dans deux mois, et l'équipe va simplement redécouvrir les mêmes arbitrages.

3. Email-Driven Architecture Antipattern

Ici, le problème n'est ni l'absence de décision, ni l'absence de justification. Le problème est que la décision et sa logique vivent dans le mauvais endroit : des fils d'emails, des messages chat, des comptes-rendus de réunion ou des slides, au lieu d'un dépôt durable et consultable.

C'est particulièrement dangereux parce que l'email donne l'impression qu'une information a été communiquée, alors qu'il constitue un très mauvais **système d'enregistrement de référence**.

La découvrabilité est faible, le versioning est mauvais, la diffusion est fragmentée, et la mémoire institutionnelle est quasi nulle. Un sous-groupe voit le message, un autre le manque, un troisième ne lit que le premier email mais pas la correction, et six mois plus tard plus personne n'est capable de reconstituer la décision finale. C'est comme cela que la dérive architecturale se produit : non pas parce que l'équipe a volontairement rejeté l'architecture, mais parce qu'elle a stocké l'architecture dans un support qui se dissipe.

La bonne pratique est simple : ne jamais considérer le corps d'un email comme l'artefact de référence. La décision doit être enregistrée dans un référentiel durable — par exemple une collection d'ADR dans le dépôt Git, un wiki d'architecture ou une base de connaissances technique — puis l'email ou le chat ne doivent servir qu'à notifier les personnes concernées et à pointer vers cette source de vérité.

- **Exemple**
L'architecte décide que tous les événements d'intégration inter-services doivent être publiés via Kafka avec des schémas Avro et des règles de rétrocompatibilité. Si cette décision vit dans une longue conversation email, l'équipe A peut la suivre, l'équipe B peut n'avoir lu qu'une version partielle et continuer avec du JSON, et l'équipe C peut ne même pas savoir qu'une décision a été prise. Si, au contraire, la règle est stockée dans un ADR lié au dépôt et référencé dans l'onboarding et les revues de code, son application devient réaliste.

La différence entre les trois antipatterns est donc simple :

- **Covering Your Assets** : l'équipe ne prend pas la décision.
- **Groundhog Day** : l'équipe a pris la décision, mais n'a pas correctement documenté la justification.
- **Email-Driven Architecture** : l'équipe a pris la décision, éventuellement justifiée, mais ne l'a pas stockée dans un endroit durable, centralisé et consultable.

Les Décisions Architecturalement Significatives

Beaucoup d'architectes considèrent qu'une décision liée à une technologie précise relève uniquement de la technique et non de l'architecture. Cette idée est incomplète. Une décision technologique peut parfaitement être une décision architecturale lorsqu'elle influence directement une qualité importante du système, comme la performance, la scalabilité, la fiabilité ou la maintenabilité.

On parle de **décision architecturalement significative** lorsqu'une décision affecte l'un des éléments suivants : la structure du système, ses caractéristiques non fonctionnelles, ses dépendances, ses interfaces ou ses techniques de construction.

comme on s'est dit, " Les décisions **architecturalement significatives** sont celles qui affectent la structure d'un système, ses caractéristiques non fonctionnelles, ses dépendances, ses interfaces ou ses techniques de construction".

- **La structure du système** concerne les choix qui influencent l'organisation globale de l'architecture. Cela inclut par exemple le style architectural adopté, comme le monolithe, les microservices, l'architecture en couches ou l'architecture orientée événements.
 - Si un architecte décide de mutualiser du code entre plusieurs microservices, ce choix peut modifier les frontières entre services (bounded context), donc les *bounded contexts*, et a alors un impact direct sur la structure du système.
- **Les caractéristiques non fonctionnelles** correspondent aux qualités attendues du système en dehors des fonctionnalités métier elles-mêmes. On parle ici de performance, sécurité, disponibilité, scalabilité, résilience, testabilité, maintenabilité, etc.
 - Par exemple, choisir une technologie parce qu'elle améliore fortement les performances d'une application fait de ce choix une décision architecturale, si la performance est un enjeu central du système.
- **Les dépendances** désignent les liens et points de couplage entre les composants ou les services. Chaque dépendance influence la manière dont les parties du système interagissent entre elles.
 - Un mauvais choix de dépendance peut nuire à la modularité, ralentir les évolutions, compliquer les tests ou réduire la fiabilité. C'est pourquoi les décisions sur les dépendances sont souvent des décisions d'architecture.
- **Les interfaces** concernent la façon dont les composants et services sont exposés, appelés et orchestrés. Cela passe souvent par des API, des passerelles (*gateways*), des adaptateurs, des proxies ou des bus d'intégration.
 - Décider de la forme d'une interface, de son contrat, de son versionnement ou de sa stratégie de dépréciation est architecturalement significatif, car ces choix affectent directement les consommateurs du système et la stabilité globale de l'écosystème.
- Enfin, **les techniques de construction** regroupent les choix liés aux plateformes, frameworks, outils, pipelines et parfois même aux processus de développement. Ces choix paraissent purement techniques, mais ils peuvent avoir un impact fort sur l'architecture.
 - Par exemple, un framework peut imposer une certaine structuration du code, limiter certains patterns, ou au contraire favoriser la modularité et la scalabilité.

L'idée essentielle à retenir est donc la suivante :

→ **une décision devient architecturale dès qu'elle influence de manière importante la forme, les qualités ou l'évolution du système.**

Le fait qu'elle implique une technologie ne la rend pas automatiquement "simplement technique".

Architectural Decision Records

est un document court qui capture une décision architecturale significative, son contexte et ses conséquences. Le concept a été formalisé par Michael Nygard en 2011 dans son article fondateur « Documenting Architecture Decisions ».

ADR vs documentation classique

Aspect	Documentation classique	ADR
Longueur	Des dizaines de pages	1-2 pages maximum
Portée	Système complet	Une seule décision
Mise à jour	Rare (souvent obsolète)	Immutable (on crée un nouvel ADR)
Question	« Comment ça marche ? »	« Pourquoi ce choix ? »
Audience	Nouveaux arrivants	Futurs décideurs
Durée de vie	Courte (devient obsolète)	Permanente (historique)

Pourquoi les ADR sont indispensables

Le coût de la connaissance perdue

Les décisions architecturales ont une caractéristique particulière : elles sont faciles à oublier mais coûteuses à redécouvrir.

Sans ADR: Imaginez ce scenario:

- 1. Un architecte fait un choix technique important
- 2. Il en parle en réunion, envoie un email
- 3. 18 mois plus tard, il quitte l'entreprise
- 4. Un nouveau développeur se demande « pourquoi PostgreSQL et pas MongoDB ? »
- 5. Personne ne se souvient, l'email est introuvable
- 6. L'équipe passe des semaines à analyser, benchmarker
- 7. Ils prennent une décision... peut-être la même, peut-être différente
- 8. Le cycle recommence

Avec ADR:

- 1. Un architecte fait un choix technique important
- 2. Il rédige un ADR en 30 minutes
- 3. 18 mois plus tard, il quitte l'entreprise
- 4. Un nouveau développeur se demande « pourquoi PostgreSQL et pas MongoDB ? »
- 5. Il ouvre docs/adr/0015-use-postgresql-for-main-database.md
- 6. En 5 minutes, il comprend le contexte, les alternatives évaluées, la décision et les justifications techniques et métiers
- 7. Il peut soit valider que le contexte est toujours pertinent, soit créer un nouvel ADR s'il a changé

Les quatre bénéfices mesurables

Bénéfice	Impact
Onboarding accéléré	-40% de temps pour comprendre l'architecture existante
Décisions plus rapides	Évite de ré-analyser les mêmes options
Moins de « pourquoi ? »	Réduit les interruptions des seniors
Audit facilité	Traçabilité des choix pour conformité

Structure Basique (Anatomie)

Un ADR efficace suit une structure standardisée. Le template le plus populaire est MADR ([Markdown](#) Architectural Decision Records), mais le template original de Michael Nygard reste une référence.

Template original de Nygard

```
# 1. Record architecture decisions
Date: 2024-01-15

## Status
Accepted

## Context
We need to record the architectural decisions made on this project.

## Decision
We will use Architecture Decision Records, as described by Michael Nygard.

## Consequences
```

See Michael Nygard's article, linked above. For a lightweight ADR toolset, see Nat Pryce's adr-tools.

The basic structure of an ADR consists of five main sections:

- Title: numéroté, action claire (« Use X for Y »),
- Status: Proposed, Accepted, Deprecated, Superseded,
- Context: pourquoi cette décision est nécessaire,
- Decision: Ce qui a été décidé,
- and Consequences: Impacts positifs et négatifs.

Un ADR n'est jamais modifié après acceptation. Si le contexte change, créez un nouvel ADR qui référence l'ancien. Cela préserve l'historique des décisions.

Quand rédiger un ADR ?

Toutes les décisions ne méritent pas un ADR. Voici comment identifier celles qui le nécessitent.

Le test des « 5 pourquoi inversés »

Si vous pouvez répondre « oui » à au moins 3 de ces questions, un ADR est justifié :

- Coût de changement : cette décision sera-t-elle coûteuse à modifier plus tard ?
- Impact équipe : plusieurs personnes seront-elles affectées ?
- Durée de vie : cette décision sera-t-elle pertinente dans 2 ans ?
- Alternatives : avez-vous évalué plusieurs options ?
- Risque de réinterprétation : quelqu'un pourrait-il mal comprendre ce choix ?

Décision	ADR nécessaire ?	Raison
Choix du langage principal	Oui	Impact majeur, durée de vie longue
Choix du framework web	Oui	Coût de changement élevé
Stratégie de branching Git	Oui	Impact équipe, réinterprétation possible
Nom d'une variable	Non	Impact négligeable
Choix entre deux bibliothèques HTTP	Selon contexte	Dépend de la criticité
Architecture microservices vs monolithe	Absolument	Décision fondamentale

Anti-patterns à éviter

L'ADR roman

- **Symptôme** : ADR de plus de 3 pages.
- **Problème** : personne ne le lira. L'ADR perd sa fonction de référence rapide.
- **Solution** : un ADR = une décision. Si vous avez besoin de plus d'espace, c'est que vous documentez plusieurs décisions.

L'ADR sans alternatives

- **Symptôme** : « Nous avons choisi X » sans mentionner Y et Z.
- **Problème** : le lecteur ne comprend pas pourquoi X plutôt qu'autre chose.
- **Solution** : toujours documenter au moins 2-3 options évaluées, même si le choix semblait évident.

```
## Options considérées
### Option 1 : Ne rien faireMaintenir l'architecture actuelle.**Rejeté** : ne résout pas le problème de scalabilité.
### Option 2 : Solution choisie[...]
```

L'ADR post-mortem

- **Symptôme** : ADR rédigé 6 mois après la décision.
- **Problème** : le contexte est oublié, les alternatives ne sont plus claires.
- **Solution** : rédiger l'ADR **au moment de la décision**, même en version draft.

L'ADR orphelin

- **Symptôme** : ADR jamais référencé, non intégré au workflow.
- **Problème** : les développeurs ne savent pas qu'il existe.
- **Solution** : intégrer les ADR au processus de revue de code, les lier dans les README.

L'ADR trop technique

- **Symptôme** : ADR incompréhensible pour quelqu'un hors de l'équipe immédiate.
- **Problème** : perd sa valeur de transmission de connaissance.

- **Solution** : un bon ADR doit être compréhensible par un développeur rejoignant l'équipe.

Processus de prise de décision architecturale

La décision est prise en suivant un processus de discussions, d'expérimentations puis de décision. Une décision peut éventuellement rendre caduque une décision passée. Cependant, ne changez pas la décision précédente, mettez-la à jour. Soit en continuant d'écrire à la suite du fichier, soit en créant un nouveau fichier qui fait référence à la décision précédente et en ajoutant un statut "Déprécié" sur la décision qui n'est plus valable.

Dans ce diagramme, nous avons intégré une étape de prise de conseils auprès d'architectes afin qu'ils aiguillent l'équipe vers les meilleurs choix en fonction de leur expérience, des projets passés et des standards de l'organisation.

```
<img src="## Architectural Decision Records
```

est un document court qui capture une décision architecturale significative, son contexte et ses conséquences. Le concept a été formalisé par Michael Nygard en 2011 dans son article fondateur « Documenting Architecture Decisions ».

ADR vs documentation classique

Aspect	Documentation classique	ADR
Longueur	Des dizaines de pages	1-2 pages maximum
Portée	Système complet	Une seule décision
Mise à jour	Rare (souvent obsolète)	Immutable (on crée un nouvel ADR)
Question	« Comment ça marche ? »	« Pourquoi ce choix ? »
Audience	Nouveaux arrivants	Futurs décideurs
Durée de vie	Courte (devient obsolète)	Permanente (historique)

Pourquoi les ADR sont indispensables

Le coût de la connaissance perdue

Les décisions architecturales ont une caractéristique particulière : elles sont faciles à oublier mais coûteuses à redécouvrir.

Sans ADR: Imaginez ce scenario:

1. Un architecte fait un choix technique important
2. Il en parle en réunion, envoie un email
3. 18 mois plus tard, il quitte l'entreprise
4. Un nouveau développeur se demande « pourquoi PostgreSQL et pas MongoDB ? »
5. Personne ne se souvient, l'email est introuvable
6. L'équipe passe des semaines à analyser, benchmarker
7. Ils prennent une décision... peut-être la même, peut-être différente
8. Le cycle recommence

Avec ADR:

1. Un architecte fait un choix technique important
2. Il rédige un ADR en 30 minutes
3. 18 mois plus tard, il quitte l'entreprise
4. Un nouveau développeur se demande « pourquoi PostgreSQL et pas MongoDB ? »
5. Il ouvre docs/adr/0015-use-postgresql-for-main-database.md
6. En 5 minutes, il comprend le contexte, les alternatives évaluées, la décision et les justifications techniques et métiers
7. Il peut soit valider que le contexte est toujours pertinent, soit créer un nouvel ADR s'il a changé

Les quatre bénéfices mesurables

Bénéfice	Impact
Onboarding accéléré	-40% de temps pour comprendre l'architecture existante
Décisions plus rapides	Évite de ré-analyser les mêmes options
Moins de « pourquoi ? »	Réduit les interruptions des seniors
Audit facilité	Traçabilité des choix pour conformité

Structure Basique (Anatomie)

Un ADR efficace suit une structure standardisée. Le template le plus populaire est **MADR** ([Markdown](#) Architectural Decision Records), mais le template original de Michael Nygard reste une référence.

```
# 1. Record architecture decisions
Date: 2024-01-15

## Status
Accepted

## Context
We need to record the architectural decisions made on this project.

## Decision
We will use Architecture Decision Records, as described by Michael Nygard.

## Consequences
See Michael Nygard's article, linked above. For a lightweight ADR toolset,
see Nat Pryce's adr-tools.
```

The basic structure of an ADR consists of five main sections:

- Title: numéroté, action claire (« Use X for Y »),
- Status: Proposed, Accepted, Deprecated, Superseded,
- Context: pourquoi cette décision est nécessaire,
- Decision: Ce qui a été décidé,
- and Consequences: Impacts positifs et négatifs.

Un ADR n'est jamais modifié après acceptation. Si le contexte change, créez un nouvel ADR qui référence l'ancien. Cela préserve l'historique des décisions.

Quand rédiger un ADR ?

Toutes les décisions ne méritent pas un ADR. Voici comment identifier celles qui le nécessitent.

Le test des « 5 pourquoi inversés »

Si vous pouvez répondre « oui » à au moins 3 de ces questions, un ADR est justifié :

- Coût de changement : cette décision sera-t-elle coûteuse à modifier plus tard ?
- Impact équipe : plusieurs personnes seront-elles affectées ?
- Durée de vie : cette décision sera-t-elle pertinente dans 2 ans ?
- Alternatives : avez-vous évalué plusieurs options ?
- Risque de réinterprétation : quelqu'un pourrait-il mal comprendre ce choix ?

Décision	ADR nécessaire ?	Raison
Choix du langage principal	Oui	Impact majeur, durée de vie longue
Choix du framework web	Oui	Coût de changement élevé
Stratégie de branching Git	Oui	Impact équipe, réinterprétation possible
Nom d'une variable	Non	Impact négligeable
Choix entre deux bibliothèques HTTP	Selon contexte	Dépend de la criticité
Architecture microservices vs monolithe	Absolument	Décision fondamentale

Anti-patterns à éviter

L'ADR roman

- **Symptôme** : ADR de plus de 3 pages.
- **Problème** : personne ne le lira. L'ADR perd sa fonction de référence rapide.
- **Solution** : un ADR = une décision. Si vous avez besoin de plus d'espace, c'est que vous documentez plusieurs décisions.

L'ADR sans alternatives

- **Symptôme** : « Nous avons choisi X » sans mentionner Y et Z.
- **Problème** : le lecteur ne comprend pas pourquoi X plutôt qu'autre chose.
- **Solution** : toujours documenter au moins 2-3 options évaluées, même si le choix semblait évident.

```
## Options considérées
### Option 1 : Ne rien faireMaintenir l'architecture actuelle.**Rejeté** : ne résout pas le problème de scalabilité.
### Option 2 : Solution choisie[...]
```

L'ADR post-mortem

- **Symptôme** : ADR rédigé 6 mois après la décision.
- **Problème** : le contexte est oublié, les alternatives ne sont plus claires.
- **Solution** : rédiger l'ADR **au moment de la décision**, même en version draft.

L'ADR orphelin

- **Symptôme** : ADR jamais référencé, non intégré au workflow.
- **Problème** : les développeurs ne savent pas qu'il existe.
- **Solution** : intégrer les ADR au processus de revue de code, les lier dans les README.

L'ADR trop technique

- **Symptôme** : ADR incompréhensible pour quelqu'un hors de l'équipe immédiate.
- **Problème** : perd sa valeur de transmission de connaissance.
- **Solution** : un bon ADR doit être compréhensible par un développeur rejoignant l'équipe.

Processus de prise de décision architecturale

La décision est prise en suivant un processus de discussions, d'expérimentations puis de décision. Une décision peut éventuellement rendre caduque une décision passée. Cependant, ne changez pas la décision précédente, mettez-la à jour. Soit en continuant d'écrire à la suite du fichier, soit en créant un nouveau fichier qui fait référence à la décision précédente et en ajoutant un statut "Déprécié" sur la décision qui n'est plus valable.

Dans ce diagramme, nous avons intégré une étape de prise de conseils auprès d'architectes afin qu'ils aiguillent l'équipe vers les meilleurs choix en fonction de leur expérience, des projets passés et des standards de l'organisation.

